Step-Indexed Hoare Logic as Ramified Forcing

Anonymous Author(s)

Abstract

We show that step-indexed Hoare logic is, conceptually and technically, *ramified forcing* — Cohen's original forcing construction stratified by a well-founded hierarchy of levels. The "later" modality ▷ is ramification's level-descent operator; Löb's rule, the central technical device of step-indexed recursive reasoning, is precisely well-founded induction on the forcing order, internalized into the logic. We make the correspondence precise via a Rocq mechanization that develops the forcing-style propositional semantics, builds Hoare logic over IMP and over an untyped λ-calculus with general recursion, gives an abstract framework parametric in the forcing notion, proves that the well-foundedness is essential by exhibiting a non-well-founded counterexample over $\mathbb{Z}$ where Löb's rule fails, and formalizes the bridge to the Jaber–Tabareau–Sozeau forcing translation of CIC via a Heyting-algebra-homomorphism embedding `Prop` $\hookrightarrow$ `iProp`. A central methodological observation is that for higher-order recursion the natural soundness proof goes through the internal Löb rule directly, whereas for first-order IMP the same content can be obtained either via the internal rule or via meta-level induction on the step index. We mechanize both forms side by side to make the equivalence concrete. The complete development is approximately 800 lines of Rocq across 11 files, with no external dependencies.

CCS Concepts: • **Theory of computation** → **Modal and temporal logics**; **Program semantics**; **Hoare logic**; *Type theory*.

Keywords: step-indexed semantics, forcing, ramified forcing, Hoare logic, Löb's rule, Rocq mechanization, Gödel–Löb logic, forcing translations

1 Introduction

Step-indexed reasoning has become the standard semantic technique for verifying higher-order programming languages with mutable state, recursive types, and concurrency. From Appel and McAllester's original step-indexed model of recursive types [Appel and McAllester 2001] to the modern Iris ecosystem [Jung et al. 2018], the same basic idea recurs: instead of asking whether a program satisfies a specification, ask whether it satisfies the specification *at every finite observation depth n*, then recover the unindexed statement as the limit. Each level of the hierarchy of approximations is defined without circularity by appeal only to lower levels, and a "later" modality ▷ mediates the descent from one level to the next.

What the step-indexed literature does not usually say is that this technique is, in classical set-theoretic terms, *forcing*. Cohen introduced forcing in 1963 to prove the independence of the continuum hypothesis [Cohen 1963, 1966]. In its original presentation — called *ramified forcing* after the ramified type theory of Russell and Whitehead [Russell and Whitehead 1913] — the construction of the generic extension is stratified by a well-founded hierarchy of *levels*, with recursion at level α permitted to consult only levels strictly below α. Shoenfield's 1971 reformulation [Shoenfield 1971] eliminated the ramification for the set-theoretic application: the modern textbook presentation of forcing does not use it. But the structure — well-founded stratification, level descent, a forcing relation $p \Vdash \varphi$ that is monotone in the strength of p — is exactly the structure of step-indexed logic.

In this paper we make the correspondence precise. The forcing *conditions* of step-indexed Hoare logic are the natural numbers ω with the order $\leq$; the forcing *relation* $n \Vdash \varphi$ is monotone in n (downward closure); the *later* modality ▷ is the level descent operator of ramified forcing; and Löb's rule, which underwrites all recursive reasoning in step-indexed systems, is precisely well-founded induction on the forcing order, internalized into the logic.

Why does this matter? The forcing reading is not just a re-labelling. It clarifies three things that are otherwise scattered across the step-indexed literature.

First, it explains *why* the technique has the shape it does. The well-founded stratification is not a presentation convenience to be eliminated; it is constitutive of the soundness of the modal logic. Section 7 makes this precise: we exhibit a non-well-founded forcing notion (the integers $\mathbb{Z}$) where every other piece of the framework goes through, but Löb's rule fails.

Second, it diagnoses where the internal Löb rule earns its keep. Section 5 compares two mechanized soundness proofs of recursive Hoare rules: the `while` rule for IMP (Section 3) and the `fix` rule for the untyped λ-calculus (Section 4). The first admits an unproblematic meta-level proof by induction on the step index; the second does not, and the natural proof invokes the internal Löb rule directly. The structural reason — that the soundness predicate is itself recursive in the `fix` case — is made precise by the forcing reading.

Third, it identifies step-indexed Hoare logic as the propositional fragment of an existing construction in the type-theory literature: the Jaber–Tabareau–Sozeau forcing translation of CIC [Jaber et al. 2012]. Section 8 formalizes this identification via a constant embedding `Prop` $\hookrightarrow$ `iProp` that is a Heyting algebra homomorphism, with ▷ appearing as the step-indexed-specific addition.

Contributions.

1. A mechanized re-presentation of step-indexed Hoare logic as ramified forcing, in Rocq. The propositional layer (Section 2) gives a forcing semantics for an intuitionistic logic with the later modality and Löb's rule. The Hoare layer is constructed over IMP (Section 3) and over an untyped λ-calculus with general recursion (Section 4).
2. An explicit comparison of the IMP `while` rule and the λ-calculus `fix` rule that diagnoses where the internal Löb rule is essential (Section 5). For IMP, soundness goes through either via meta-level induction on the step index or via the internal Löb rule; we exhibit both proofs and observe their structural identity. For the λ-calculus with `fix`, the natural soundness proof must invoke the internal Löb rule, because the soundness predicate is itself recursive in the program. This is the strongest piece of evidence for the slogan that "Löb is what makes step-indexed Hoare logic work in higher-order settings."
3. An abstract forcing framework (Section 6) parametric over an arbitrary well-founded forcing structure. We exhibit two instances: $(\omega, \leq)$, recovering the standard step-indexed semantics; and $(\omega \times \omega, \leq_{\mathrm{pw}})$, with the pointwise order, modelling independent step-indices for parallel components. The internal Löb rule, in the abstract setting, is proved by `well_founded_ind` on the strict refinement order. This makes precise the equivalence: *internal Löb is well-founded induction on the forcing order, internalized.*
4. A small new metatheoretic result: *well-foundedness of the forcing notion is essential* (Section 7). We exhibit a non-well-founded "would-be" forcing structure over $\mathbb{Z}$, in which every iProp construction goes through except Löb's rule. The premise of Löb's rule $(\triangleright\bot) \to \bot$ is forced at every condition — vacuously, since $\mathbb{Z}$ has no minimum — while $\bot$ is forced at none. This is a special case of the model theory of the Gödel–Löb provability logic [Boolos 1993; Solovay 1976], brought into the step-indexed Hoare logic setting and mechanized.
5. A partial bridge to the Jaber–Tabareau–Sozeau forcing translation of CIC [Jaber et al. 2012] (Section 8). We formalize the constant embedding Prop $\hookrightarrow$ `iProp` as a Heyting algebra homomorphism, and observe that the $\triangleright$ modality is the genuinely new content of the iProp universe over the embedded image of Prop. A full mechanization of the type-level JTS translation is future work.

What is and is not new. The individual ingredients are largely known to specialists in neighbouring fields. The topos-of-trees perspective on step-indexing [Birkedal et al. 2012] is implicitly a forcing semantics; the Gödel–Löb modal logic of provability is the modal logic of provability and well-foundedness; the JTS forcing translation of CIC is the type-theoretic analog of forcing. What is new is the assembly: a single mechanized account that makes the forcing reading explicit, diagnoses the role of the internal Löb rule, and gives a parametric framework with the metatheoretic guarantee that well-foundedness cannot be relaxed.

Outline. Section 2 gives the propositional layer of the forcing semantics, including the Heyting-algebra connectives, the later modality, and Löb's rule. Section 3 develops Hoare logic for IMP and proves the `while` rule by meta-level induction; Section 4 develops Hoare logic for an untyped λ-calculus with `fix` and proves the soundness rule for `fix` via the internal Löb rule. Section 5 compares these two proofs explicitly and articulates the diagnostic claim. Section 6 gives the abstract forcing framework. Section 7 proves Löb's rule fails without well-foundedness. Section 8 establishes the constant-embedding bridge to JTS forcing translations. Section 9 positions the contribution against the literature. Section 10 concludes.

Rocq development. The complete Rocq development is approximately 800 lines across 11 files, building cleanly against the Rocq standard library only (no external libraries such as Iris or stdpp).

2 The Propositional Layer

Before defining a Hoare logic we need a propositional logic to express the pre- and post-conditions in. To make explicit the connection to forcing, we build this propositional logic as a Kripke-style forcing semantics: a proposition is not a single truth value but a *family* of truth values indexed over a poset of *conditions*, and entailment is forcing at every condition.

The simplest forcing notion that supports step-indexed reasoning is the natural numbers with the usual order. Each natural number stands for an "observation depth": condition n records that we have consumed n program steps and committed to a partial view of the computation. Following the topos-of-trees and Iris convention, we take *smaller* naturals to be *stronger* conditions: 0 is the weakest condition (we have observed nothing and are committed to nothing), and larger n are stronger (we are committed to a deeper approximation). Under this orientation, the forcing extension order is the converse of $\leq_{\mathbb{N}}$, and a forcing-style proposition must be *downward closed* in $\leq_{\mathbb{N}}$: a proposition forced at depth n is forced at every shallower depth $m \leq n$.

```
Definition cond : Type := nat.

Record iProp : Type := MkProp {
  ihold : cond → Prop;
  imono : forall n m, m ≤ n → ihold n →
       ihold m
}.

Notation "n ⊩ φ" := (ihold φ n).
```

The notation $n \Vdash \varphi$ matches the classical forcing notation, and it is more than analogical: an `iProp`, taken with its `imono` field, is exactly a presheaf on $(\omega, \leq)$, hence an object of the topos of trees [Birkedal et al. 2012] — which is exactly the Kripke model of intuitionistic logic with worlds in ω, and hence (Section 9.3) the forcing-style semantics with conditions in ω.

Entailment is forcing at every condition:

```
Definition ientails (φ ψ : iProp) : Prop
    :=
  forall n, n ⊩ φ → n ⊩ ψ.

Notation "φ ⊢ ψ" := (ientails φ ψ).
```

Connectives. The Heyting-algebra connectives have the standard Kripke interpretation. Truth and falsity are the constant predicates; conjunction and disjunction are pointwise; implication is the Kripke implication, quantified over all stronger conditions:

```
Definition iAnd (φ ψ : iProp) : iProp :=
  MkProp (fun n => n ⊩ φ /\ n ⊩ ψ) (
      iAnd_mono φ ψ).

Definition iImpl (φ ψ : iProp) : iProp :=
  MkProp (fun n => forall m, m ≤ n → m ⊩
       φ → m ⊩ ψ)
          (iImpl_mono φ ψ).
```

The quantification "$\forall m \leq n$" in `iImpl` is the characteristic move of intuitionistic Kripke semantics; without it, the framework would collapse to classical logic with a step index. The monotonicity proofs `iAnd_mono` and `iImpl_mono` are routine; in each case the proof simply transports the membership witness through the order.

The later modality. The genuinely new ingredient over the standard Kripke interpretation of intuitionistic logic is the *later* modality $\triangleright$. It is the operator that says "to talk about a proposition here, you must drop one level". In our $(\omega, \leq)$ orientation, $\triangleright\varphi$ at depth n holds if $n = 0$ (we are at the weakest condition and the modality is vacuously satisfied) or if φ holds at the shallower depth $n - 1$:

```
Definition iLater (φ : iProp) : iProp :=
  MkProp (fun n => match n with
                   | 0 => True
                   | S k => k ⊩ φ
                   end) (
                       iLater_mono_helper φ
                       ).

Notation "▷ φ" := (iLater φ).
```

Two readings of this operator are worth stating explicitly. In step-indexed-logic terms, $\triangleright\varphi$ is "φ when we look at the computation one step later", and the shift is what guards self-referential definitions against circularity. In forcing terms, $\triangleright$ is the explicit level-descent operator: at level $n+1$ we may consult level n. This is precisely the ramification of Cohen's original construction (Section 9, "Forcing in set theory"): the recursive definition of forcing at level α may invoke forcing only at strictly lower levels, and $\triangleright$ is the syntactic device that makes the descent explicit.

Heyting algebra rules. With the connectives in hand, the standard intuitionistic introduction and elimination rules become direct consequences of their Kripke definitions. We mechanize the full set:

```
Lemma iAnd_intro φ ψ χ : φ ⊢ ψ → φ ⊢ χ →
    φ ⊢ ψ ∧ χ.
Lemma iAnd_proj_l φ ψ : φ ∧ ψ ⊢ φ.
Lemma iOr_intro_l φ ψ : φ ⊢ φ ∨ ψ.
Lemma iOr_elim φ ψ χ : φ ⊢ χ → ψ ⊢ χ → φ
    ∨ ψ ⊢ χ.
Lemma iImpl_intro φ ψ χ : φ ∧ ψ ⊢ χ → φ ⊢
    (ψ → χ).
Lemma iImpl_elim φ ψ : (φ → ψ) ∧ φ ⊢ ψ.
```

Each proof simply moves the assumed monotonicity through the relevant connective; the work is small but worth doing mechanically because it confirms that the framework is genuinely a Heyting algebra without any informal appeal to topos-theoretic abstractions.

The basic laws of the later modality follow the same pattern. We prove that $\triangleright$ is monotone in the entailment order, that any proposition entails its own later ($\varphi \vdash \triangleright\varphi$), that $\triangleright$ distributes over $\wedge$ in both directions, and that $\triangleright\top$ is interderivable with $\top$.

Löb's rule. The technical heart of the propositional layer is Löb's rule:

Theorem 1 (Löb's rule)**.** *For every* `iProp` φ, $(\triangleright\varphi \to \varphi) \vdash \varphi$.

We give the rule the standard step-indexed proof, by induction on n:

```
Lemma iLob φ : (▷ φ → φ) ⊢ φ.
Proof.
  intros n Hn.
  induction n as [|n IH].
  - apply (Hn 0); [lia | exact I].
  - apply (Hn (S n)); [lia | simpl].
    apply IH.
    intros m Hm Hlate.
    apply (Hn m); [lia | exact Hlate].
Qed.
```

It is worth noticing what this proof actually uses. The base case relies on the fact that $\triangleright\varphi$ at $n = 0$ is *vacuously* `True`; the inductive step uses the hypothesis at $n+1$ together with the induction hypothesis at n. In other words, Löb's

rule is, at the meta level, well-founded induction on the forcing order ω. This observation will return in Section 6: we will see there that in the abstract framework Löb's rule *is* `well_founded_ind` on the strict refinement relation.

Step-indexing is not collapsing. A useful sanity check is that the later modality changes the propositional theory: it is not the case that $\triangleright\bot$ entails $\bot$. At $n = 0$, the proposition $\triangleright\bot$ is `True` by definition, whereas $\bot$ is `False`, so the entailment fails at $n = 0$:

```
Lemma iLater_False_distinct : ~ (▷ ⊥ ⊢ ⊥).
Proof. intros H. apply (H 0); simpl; exact
    I. Qed.
```

This is the canonical witness that step-indexing genuinely extends the propositional theory, rather than collapsing to ordinary intuitionistic logic. We will revisit it in Section 7 to make the converse point: if we *drop* the well-foundedness of the forcing order, then $\triangleright\bot$ becomes equivalent to $\bot$ in a way that breaks Löb's rule entirely.

3 Hoare Logic for IMP

With the propositional layer in hand we can build a Hoare logic for a simple imperative language. We do this in the standard calibrating setting — a small while-language with mutable state, which we will call IMP — because the conceptual point of this section is most cleanly visible there. IMP has no higher-order features, no allocation, and no concurrency, so the step indices we attach to assertions are not yet doing technical work. What the section accomplishes is showing how the forcing-style propositional layer of Section 2 lifts to a sound Hoare logic, and — crucially for what follows — proving the `while` rule both by meta-level induction on the step index and by an explicit appeal to the internal Löb rule of Section 2. These two proofs establish the same theorem, and exhibiting them side by side makes precise the slogan that "Löb's rule *is* induction on the step index".

IMP and its semantics. The language is standard: arithmetic expressions over an integer state, boolean expressions, and the usual control-flow commands. Variables are natural numbers and the state is a function `var -> Z`.

```
Inductive cmd : Type :=
  | CSkip   : cmd
  | CAssign : var → aexp → cmd
  | CSeq    : cmd → cmd → cmd
  | CIf     : bexp → cmd → cmd → cmd
  | CWhile  : bexp → cmd → cmd.
```

We give a small-step operational semantics `step : cmd * state -> cmd * state -> Prop` with the usual rules: assignment updates the state and reduces to `CSkip`; sequencing reduces its left component if not yet `CSkip` and otherwise advances to its right component; conditionals branch on the boolean evaluation; `while` either unfolds to a sequence or terminates as `CSkip`. Each step in this semantics corresponds to one decrement of the step-index clock that the wp predicate is about to track.

Step-indexed weakest precondition. The wp predicate for a command is a step-indexed analogue of the classical Dijkstra wp: `wp_at` $n\,c\,Q\,s$ says that if we run c from state s for at most n small steps, no intermediate configuration is bad and any final state satisfies the postcondition Q. Because the only "bad" configuration in IMP is one that is neither `CSkip` nor able to step — and in IMP every non-`CSkip` command can step — the safety condition is automatic, and we do not need to record it explicitly. This invisibility of safety in IMP is what makes the section a calibration; in Section 4 we will see what happens when the language is rich enough for stuck terms to exist.

```
Fixpoint wp_at (n : nat) (c : cmd)
               (Q : state → Prop) (s :
                  state) : Prop :=
  match n with
  | O => True
  | S k =>
      (c = CSkip → Q s) /\
      (forall c' s', step (c, s) (c', s')
          → wp_at k c' Q s')
  end.
```

The predicate is downward closed in n (`wp_at_mono`) and contravariantly monotone in the postcondition Q (`wp_at_post_mono`); both proofs are routine inductions on n. Lifting to an `iProp` is then immediate:

```
Definition wp (c : cmd) (Q : state → Prop
    ) (s : state) : iProp :=
  MkProp (fun n => wp_at n c Q s) (
      wp_at_mono c Q s).
```

The Hoare triple. A Hoare triple is the `iProp` that says "for every state satisfying P, the wp of c targeting Q holds". By the universal quantifier, this is again a downward-closed predicate in n:

```
Definition hoare (P : state → Prop) (c :
    cmd)
               (Q : state → Prop) :
                  iProp :=
  MkProp (fun n => forall s, P s → wp_at
      n c Q s)
         (hoare_at_mono P c Q).

Definition hoare_valid P c Q : Prop := ⊤ ⊢
     hoare P c Q.
```

We say the triple is *valid* when it is forced at every condition: $\top \vdash$ `hoare` $P\,c\,Q$, which by unfolding is exactly the standard partial-correctness statement "for every n and every

state s with $P\,s$, the n-step-bounded wp holds". This equivalence is the lemma `hoare_valid_alt` in the development; we will use it freely to move between the iProp-internal and the meta-level views of a Hoare triple.

Soundness of the structural rules. The standard structural rules of partial-correctness Hoare logic become small lemmas about `wp_at`. We state them in the validity form:

```
Lemma hoare_skip P : hoare_valid P CSkip P
    .
Lemma hoare_assign Q x a :
  hoare_valid (asgn_pre Q x a) (CAssign x
      a) Q.
Lemma hoare_seq P c1 R c2 Q :
  hoare_valid P c1 R →
  hoare_valid R c2 Q →
  hoare_valid P (CSeq c1 c2) Q.
Lemma hoare_if P b c1 c2 Q :
  hoare_valid (fun s => P s /\ beval s b =
       true) c1 Q →
  hoare_valid (fun s => P s /\ beval s b =
       false) c2 Q →
  hoare_valid P (CIf b c1 c2) Q.
Lemma hoare_consequence P P' c Q Q' :
  (forall s, P s → P' s) → hoare_valid P
      ' c Q' →
  (forall s, Q' s → Q s) → hoare_valid P
       c Q.
```

Each proof is a routine unfolding of the wp predicate at $n+1$, followed by inversion on the available step. Sequencing requires a small auxiliary lemma `wp_seq` that decomposes the wp of `CSeq c1 c2` as the wp of `c1` targeting the wp of `c2`; this is the only point where the step index moves between the outer and inner wp, and a use of `wp_at_mono` is needed to align the indices. None of this requires the propositional layer of Section 2; we could have written the same five rules entirely at the meta level, with `hoare_valid` as a plain "forall n, s" statement. The propositional layer is present because it will pay off in the next part of this section — and again, more decisively, in Section 4.

The `while` rule: meta-level induction. The interesting rule is `while`. Soundness of

$$\frac{\{P \wedge b\}\, c\, \{P\}}{\{P\}\, \mathsf{while}\, b\, \mathsf{do}\, c\, \{P \wedge \neg b\}}$$

is, in step-indexed terms, the statement that a sound loop body gives a sound loop — but the loop's wp at $n+1$ unfolds into a proof obligation about the loop's wp at n, which is exactly the recursive structure we have been preparing for. The straightforward proof is by induction on n:

```
Lemma hoare_while P b c :
  hoare_valid (fun s => P s /\ beval s b =
       true) c P →
  hoare_valid P (CWhile b c)
              (fun s => P s /\ beval s b =
                   false).
Proof.
  intros HC.
  rewrite hoare_valid_alt in HC.
  apply hoare_valid_alt.
  intros n. induction n as [|n IH]; intros
       s HP.
  - simpl. trivial.
  - rewrite wp_at_S. split.
    + discriminate.
    + intros c' s' Hstep. inversion Hstep;
         subst.
      * (* StepWhileTrue *)
        apply wp_seq.
        eapply wp_at_post_mono.
        2: { apply HC; split; [exact HP |
            assumption]. }
        intros s'' HP'. apply IH. exact HP
            '.
      * (* StepWhileFalse *)
        ... (* CSkip with the
            postcondition *)
Qed.
```

The structure is what one would write in any step-indexed Hoare logic. The case analysis on n contributes a trivial base case (the wp at 0 is `True`) and an inductive step. In the `StepWhileTrue` case, the loop reduces to c; `while` $b\,c$ at step n, and the proof obligation for the remaining `while` $b\,c$ piece is discharged by the induction hypothesis `IH`.

The same rule, via the internal Löb. There is also a proof of `hoare_while` that does not use any explicit induction on n — it appeals to the internal Löb rule of Section 2 directly. To use `iLob`, we need to express the goal as an entailment $(\triangleright\varphi \to \varphi) \vdash \varphi$. This is done by an application of `ientails_trans`: we factor the desired entailment $\top \vdash \mathsf{hoare}\, P\, (\mathsf{CWhile}\, b\, c)\, Q$ through the intermediate iProp $\triangleright\varphi \to \varphi$ for $\varphi := \mathsf{hoare}\, P\, (\mathsf{CWhile}\, b\, c)\, Q$, prove the intermediate $\top \vdash \triangleright\varphi \to \varphi$ directly, and finish with `apply iLob`:

```
Lemma hoare_while_iLob P b c :
  hoare_valid (fun s => P s /\ beval s b =
       true) c P →
  hoare_valid P (CWhile b c)
              (fun s => P s /\ beval s b =
                   false).
Proof.
  intros HC. rewrite hoare_valid_alt in HC
      .
  unfold hoare_valid.
  apply ientails_trans with
    (iLater (hoare P (CWhile b c) (...)) →
```

```
      hoare P (CWhile b c) (...)).
  - (* ⊤ ⊢ ▷ φ → φ *)
    intros n _ m Hmn Hlater s HP.
    destruct m as [|m'].
    + simpl. trivial.
    + simpl in Hlater. rewrite wp_at_S.
      split.
      * discriminate.
      * intros c' s' Hstep. inversion
        Hstep; subst.
        -- (* StepWhileTrue *)
           apply wp_seq.
           eapply wp_at_post_mono.
           2: { apply HC; split; [exact HP
                | assumption]. }
           intros s'' HP''. apply Hlater.
             exact HP''.
        -- (* StepWhileFalse *)
           ...
  - apply iLob.
Qed.
```

The body of the first bullet — the proof that $\top$ entails $\triangleright\varphi \to \varphi$ at every condition — contains no induction on n. We case-split on the depth m obtained from the iImpl, find $\triangleright\varphi$ supplied to us at depth m, and in the $\mathsf{S}\,m'$ case use it (it is φ at depth m') to discharge the recursive obligation. All of the recursive content is hidden inside `iLob`'s own proof, which of course does eventually go by induction on n — but *once*, in Section 2, and not at every Hoare-style soundness proof.

The packaged soundness theorem. With both proofs available, we package the six structural rules into a single theorem statement:

```
Theorem imp_hoare_rules :
  (forall P, hoare_valid P CSkip P) /\
  (forall Q x a, hoare_valid (asgn_pre Q x
      a) (CAssign x a) Q) /\
  (forall P c1 R c2 Q, ... → ... → ...)
     /\  (* seq *)
  (forall P b c1 c2 Q, ... → ... → ...)
     /\  (* if *)
  (forall P b c, ... → ...) /\
                  (* while *)
  (forall P P' c Q Q', ... → ... → ...
     → ...).  (* consequence *)
```

This is the partial-correctness soundness statement for Hoare logic over IMP, in the forcing-style semantics. It is structurally identical to the soundness statement one would prove for IMP using the standard direct semantics, and that identity is part of the point: at the first-order level the forcing reading is a different *presentation* of the same theorems. In Section 4 we will move to a setting where the re-presentation produces a noticeably different proof obligation.

4 Hoare Logic for an Untyped λ-Calculus with Recursion

For IMP, step-indexing was overkill: the wp predicate could have been defined without indexing at all, and the index was a presentation convenience. We now move to a language where step-indexing is no longer optional — an untyped λ-calculus with a fix-point operator. Two things change. First, the wp predicate must explicitly encode *safety*: applying an integer as a function, or branching on a lambda, leaves the computation stuck, and we want the wp to rule such configurations out rather than treat them as vacuously good. Second, the recursion introduced by the fix-point operator turns the soundness predicate for a recursive function into something defined in terms of itself, and the natural soundness proof for the corresponding Hoare rule no longer goes through meta-level induction on the step index. It goes through an appeal to the internal Löb rule of Section 2, used directly, with no outer induction on n.

The language. We use a de-Bruijn representation, with integer constants for the sake of having a non-trivial base type, an `if0` conditional that branches on whether its argument equals zero, λ-abstraction and application, and an explicit fix-point binder:

```
Inductive expr : Type :=
  | EVar  : nat → expr
  | EInt  : Z → expr
  | EPlus : expr → expr → expr
  | EIf0  : expr → expr → expr → expr
  | ELam  : expr → expr
  | EApp  : expr → expr → expr
  | EFix  : expr → expr.

Inductive value : expr → Prop :=
  | VInt : forall n, value (EInt n)
  | VLam : forall body, value (ELam body)
  | VFix : forall body, value (EFix body).
```

The fix-point form `EFix` *body* binds the self-reference at de-Bruijn index 0 inside *body*; we treat it as a value so that it can be passed as an argument, and we let the self-unfolding fire only when the fix is *applied*. Substitution is the standard single substitution; we omit its definition here. The small-step relation includes the usual rules for arithmetic and conditionals together with β-reduction $\mathsf{EApp}\,(\mathsf{ELam}\,body)\,v \to body[v/0]$ and the fix-unfolding rule

```
| StepFix : forall body v, value v →
    EApp (EFix body) v ~>
    EApp (subst (EFix body) body) v.
```

which substitutes the self-reference `EFix` *body* for index 0 inside the body before letting an ordinary β-reduction proceed. All reductions are call-by-value.

Step-indexed wp, with safety. The wp predicate is similar in shape to the IMP version but has an additional conjunct that pins down safety. At index $n+1$ we ask that the expression either is a value satisfying the postcondition, or is reducible and every reduct still satisfies the wp at index n:

```
Fixpoint wp_at (n : nat) (e : expr)
               (Q : expr → Prop) : Prop
                  :=
  match n with
  | O => True
  | S k =>
      (value e → Q e) /\
      (value e \/ exists e', step e e') /\
      (forall e', step e e' → wp_at k e'
          Q)
  end.
```

The middle conjunct (`value e \/ exists e', step e e'`) is the new ingredient. Without it, a stuck non-value expression such as `EApp (EInt 5) (EInt 3)` would satisfy the wp vacuously: the first conjunct is discharged (the expression is not a value), and the third quantifies over a step relation that has no inhabitants for the stuck expression. With it, the stuck expression fails the wp. This is the price of working with a language whose syntax admits ill-typed combinations; the IMP development of Section 3 did not need it because every non-`CSkip` command can step.

Basic rules. Two general structural rules carry every soundness proof. The first says that a value satisfies the wp targeting any postcondition it semantically satisfies:

```
Lemma wp_value v Q :
  value v → Q v → wp_valid v Q.
```

The second is the *pure step* rule: if every reduct of e is e' and e' satisfies the wp, then so does e.

```
Lemma wp_pure_step_at n e e' Q :
  ~ value e →
  (exists e'', step e e'') →
  (forall e'', step e e'' → e'' = e') →
  wp_at n e' Q →
  wp_at (S n) e Q.
```

The pure-step rule is what propagates the wp through any β-reduction or fix-unfolding step. In particular, it is the device by which we will discharge the fix-unfolding step in the `wp_fix` soundness proof below.

We do not develop dedicated wp rules for `EPlus`, `EIf`, or `EApp` at this stage — they either need an evaluation-context framework or unfold case-by-case via `wp_pure_step`. Neither addition is necessary for the conceptual point of this section; we will see them play out concretely in the worked example below.

The specification predicate for recursive functions. To state a meaningful soundness rule for `fix`, we need a shape of specification appropriate to a recursive function. The simplest non-trivial shape is the *closed* spec, where the same predicate S on values plays both the role of a precondition on the argument and the role of a postcondition on the result. We package this as an iProp:

```
Definition recursive_spec (e : expr) (S :
    expr → Prop) : iProp :=
  MkProp
    (fun n => forall v, value v → S v →
                          wp_at n (EApp e
                              v) S)
    (recursive_spec_mono e S).
```

The proposition `recursive_spec` e S at depth n says: "for every value v satisfying S, applying e to v is safe through n steps, and any value it produces satisfies S". Crucially, S appears *twice* — once as the precondition, once as the postcondition. This is the source of the self-reference: to prove the spec for `EFix` *body* at depth $n+1$, we need to know the spec for `EFix` *body* at depth n in order to handle the recursive call inside the body.

The `wp_fix` soundness rule. The soundness rule we prove is:

Theorem 2 (`wp_fix`, the soundness rule for fix-points)**.** *Let* body *be an expression and S a predicate. Suppose that for every value e_{self} for which* `recursive_spec` e_{self} S *holds, we also have* `recursive_spec` (`subst` e_{self} *body*) S *(a Lipschitz-like assumption on the body). Then* $\top \vdash$ `recursive_spec` (`EFix` *body*) S.

In words: if the body is a well-behaved transformer of recursive specs — if it produces a function satisfying the spec from a self-reference satisfying the spec — then the fix-point itself satisfies the spec.

The natural proof uses the internal Löb rule directly. We factor the goal $\top \vdash$ `recursive_spec` (`EFix` *body*) S through the iProp $(\triangleright\varphi \to \varphi)$ for $\varphi :=$ `recursive_spec` (`EFix` *body*) S, prove the intermediate $\top \vdash \triangleright\varphi \to \varphi$ directly — this is a finite piece of step-indexed reasoning, with no induction on n — and close with `apply iLob`:

```
Theorem wp_fix body S :
  (forall e_self, value e_self →
     recursive_spec e_self S ⊢
        recursive_spec (subst e_self body)
          S) →
  ⊤ ⊢ recursive_spec (EFix body) S.
Proof.
  intros Hbody.
  apply ientails_trans with
```

```
      (iLater (recursive_spec (EFix body) S)
          →
       recursive_spec (EFix body) S).
  - (* ⊤ ⊢ ▷ φ → φ *)
    intros n _ m Hmn Hlater.
    destruct m as [|m'].
    + intros v Hvalue HS. simpl. trivial.
    + simpl in Hlater. intros v Hvalue HS.
      apply (wp_pure_step_at m'
        (EApp (EFix body) v)
        (EApp (subst (EFix body) body) v)
            S).
      * apply not_value_app_fix.
      * apply step_fix_progress; exact
          Hvalue.
      * intros e' Hs. apply step_fix_det;
          auto.
      * apply (Hbody (EFix body) (VFix
          body) m'
                      Hlater v Hvalue HS).
  - apply iLob.
Qed.
```

It is worth spelling out what the proof actually does. The `apply ientails_trans` step factors the goal through Löb's form. In the body of the first bullet, the case analysis on m contributes only a trivial base case ($m = 0$, where the wp is `True`) and an inductive step. In the inductive case, the hypothesis `Hlater` supplies us with `recursive_spec` (`EFix` *body*) S at the *predecessor* index m' — which is exactly the data we need to satisfy the precondition of `Hbody` when discharging the recursive call. The fix-unfolding step from `EApp` (`EFix` *body*) v is then a single application of `wp_pure_step_at`, which leaves us at index m' inside the substituted body — and the substituted body satisfies the spec at m' by Hbody and Hlater.

There is no induction on n in this proof. The induction is hidden inside `iLob`'s own proof, which we wrote once in Section 2. This is the structural feature that we want to spotlight in Section 5: the soundness predicate `recursive_spec` is recursive in the function being verified, and so the natural soundness proof must invoke *some* recursive reasoning principle. In a meta-level induction we would have to manufacture that recursion from outside; the internal Löb rule has it built in.

Worked examples. We close the section with two short verifications that exercise the framework end-to-end. The first is a fix-based identity — `recursive_id` := `EFix` (`ELam` (`EVar` 0)), which observably equals the plain identity $\lambda x.x$ but forces a fix-unfolding step at every application:

```
Definition recursive_id : expr := EFix (
    ELam (EVar 0)).

Theorem recursive_id_spec (S : expr →
    Prop) :
  ⊤ ⊢ recursive_spec recursive_id S.
```

The body `ELam` (`EVar` 0) does not mention the self-reference at index 1, so substitution into it is the identity on expressions, and the Lipschitz premise of `wp_fix` becomes a non-recursive statement that we discharge directly with `wp_pure_step_at`. This example is the simplest possible exercise of the rule: it confirms that the framework can verify a recursive function whose recursive structure is trivial.

The second example is the canonical divergent fix-point:

```
Definition bottom : expr := EFix (EVar 0).

Theorem bottom_spec (S : expr → Prop) :
  ⊤ ⊢ recursive_spec bottom S.
Proof.
  apply wp_fix. intros e_self Hself.
  apply ientails_refl.
Qed.
```

For `bottom`, the body *is* the bare self-reference at index 0, so substituting e_{self} into it gives e_{self} unchanged, and the Lipschitz premise of `wp_fix` collapses to reflexivity. The conclusion — that `bottom` satisfies every recursive spec — is the familiar observation that a divergent program satisfies any partial-correctness specification vacuously; here it appears as a one-liner because the forcing framework absorbs the divergence into the wp's index hierarchy.

5 Why the Two Proofs Look Different

We have now mechanized two Hoare-style soundness theorems against the same propositional forcing semantics: the `while` rule of Section 3 and the `wp_fix` rule of Section 4. Both are statements about recursively defined programs. Both go by some form of recursion in the proof. But the proofs look structurally different, and that difference is the main technical observation we want to articulate.

The IMP proof of `hoare_while` proceeds by an explicit outer induction on the step index n. The base case discharges the wp at depth 0 (which is `True`). The inductive step unfolds the wp at depth $n + 1$, case-analyses on the step taken by `CWhile b c`, and in the recursive branch uses the induction hypothesis at depth n to discharge the obligation for `CWhile b c` itself. The proof can also be done by an appeal to the internal Löb rule, as the `hoare_while_iLob` alternative shows; but the two proofs are structurally identical, and the choice between them is cosmetic.

The λ-calculus proof of `wp_fix` also goes by some form of recursion — it has to, because the goal is the soundness of a recursive function — but it does not contain any induction on n at all. It factors the goal through Löb's form $\triangleright\varphi \to \varphi$ and lets the recursion happen inside `iLob`. The interior of the factored proof is finite, non-recursive step-indexed reasoning: a single

case analysis on the step index supplied by the `iImpl`, one unfolding of the fix-application step via `wp_pure_step_at`, and one appeal to the Lipschitz hypothesis on the body.

Why does the difference arise? The structural reason is what we call a difference in the *recursion locus* of the soundness predicate.

For `while`, the wp predicate `wp_at` n (`CWhile` $b\ c$) $Q\ s$ unfolds at depth n+1 into an obligation that mentions `wp_at` n (`CWhile` $b\ c$) $Q\ s'$ at depth n for some state s'. The proposition recursive at the lower depth is *the very same proposition*, decremented in the index. At the meta level, this is just induction on n: each level of the recursion consumes one step index and we run out of indices in finitely many steps. No specifically modal reasoning is needed because the recursive obligation is parameterised solely by n.

For `fix`, the picture is different. The proposition `recursive_spec` (`EFix` *body*) S at depth $n + 1$ unfolds, after one fix-unfolding step, into an obligation about the same `EFix` *body* at depth n — *but only because the body's behaviour at the recursive call site is guaranteed by the same proposition at depth n*. In other words, the recursive proposition appears both inside and outside the recursion: as the spec we are trying to prove for `EFix` *body*, and as the hypothesis we need at the recursive call. At the meta level, an outer induction on n can in principle reach the recursive call — one can always recurse on n — but the proof obligation at the recursive call is no longer a statement purely about smaller step indices; it is a statement about `EFix` *body* itself at a smaller step index. This is exactly the shape that Löb's rule was designed to handle: "assume the spec holds later, prove it holds now", where "later" means "at a strictly smaller condition". Internalizing the recursion into Löb's rule is what frees the proof from carrying an outer induction on n.

A diagnostic. We can state this as a working slogan:

> **If the soundness predicate is itself recursive in the program** — if its definition at one depth refers to its definition for the same program at a lower depth — then internal Löb is the natural soundness tool, and a meta-level induction on the step index fights the structure of the predicate. If the soundness predicate is only recursive in the depth, an outer induction is interchangeable with Löb and either is cheap.

The IMP `wp_at`, `hoare`, and `while` fall on the easy side of the diagnostic. The recursive-spec `recursive_spec` (`EFix` *body*) S falls on the hard side, and Löb earns its keep there.

What this says about step-indexed Hoare logic. In the verification literature, Löb's rule is sometimes presented as a convenience: a tactic available in the Iris proof mode, used when recursion is involved, but conceptually equivalent to "the step-indexed argument". Our diagnostic says this is right for first-order recursion (the IMP case) but understates what is happening in higher-order recursion (the λ-fix case). When the soundness predicate is genuinely self-referential in the program, an outer induction on n is still possible but is no longer playing to the structure of the predicate. The internal Löb rule is what plays to that structure — and it is, at bottom, well-founded induction on the forcing order, internalized into the logic so that it can act on the recursive proposition directly. Section 6 will make precise what "internalized" means by showing the abstract framework's Löb rule is literally an appeal to `well_founded_ind`.

6 An Abstract Forcing Framework

So far, the forcing notion has been the specific poset $(\omega, \leq)$. The proofs of Sections 2–4 have used the natural numbers throughout: as the type of conditions, as the index of the wp predicate, and as the well-founded order on which Löb's rule was proved. But none of these uses required ω specifically. In this section we abstract the framework over an arbitrary well-founded forcing notion, and prove Löb's rule *once* at this level of generality.

The abstraction makes precise two claims that have been hovering over the development. First, the choice of ω in Section 2 is a presentation convenience rather than a foundational commitment: any well-founded preorder will do. Second, the relationship between Löb's rule and well-founded induction that we have been describing analogically becomes a literal identity in the abstract setting — the abstract `iLob` is proved by `apply (well_founded_ind fs_lt_wf)`.

The forcing structure. We package the assumed structure into a record:

```
Record ForcingStructure : Type := MkFS {
  fs_cond : Type;
  fs_le : fs_cond → fs_cond → Prop;
  fs_lt : fs_cond → fs_cond → Prop;
  fs_le_refl : forall p, fs_le p p;
  fs_le_trans : forall p q r,
                  fs_le p q → fs_le q r
                    → fs_le p r;
  fs_lt_le : forall p q, fs_lt p q →
      fs_le p q;
  fs_lt_le_lt : forall p q r,
                  fs_lt p q → fs_le q r
                    → fs_lt p r;
  fs_lt_wf : well_founded fs_lt
}.
```

A forcing structure is a preorder (`fs_cond`, `fs_le`) together with a strict relation `fs_lt` contained in `fs_le` and compatible with `fs_le` on the right, such that `fs_lt` is well-founded. The compatibility axiom `fs_lt_le_lt` is the technical ingredient that makes the later modality well-defined in the abstract setting; we explain this below.

iProp, connectives, and* `iLater` *at the abstract level. Over an arbitrary forcing structure `FS`, the `iProp` record is the obvious adaptation: a downward-closed predicate on `fs_cond FS` with respect to `fs_le FS`. The propositional connectives are defined exactly as in Section 2, with the order $\leq$ replaced by `fs_le FS`. The interesting case is the later modality. At the specific $(\omega, \leq)$ instance, we defined $\triangleright\varphi$ at depth n by a match on the structure of n (`True` if $n = 0$, φ at $n-1$ otherwise). At an arbitrary forcing notion, we cannot match on conditions because the carrier is not a constructive inductive type. We instead give the modality a uniform definition: $\triangleright\varphi$ holds at a condition p if and only if φ holds at every strict predecessor of p:

```
Definition iLater (φ : iProp) : iProp :=
  MkProp
    (fun p => forall p', fs_lt FS p' p →
        p' ⊩ φ)
    (iLater_mono_helper φ).
```

At a *minimal* condition — one with no strict predecessors — this is vacuously `True`. At a successor condition, it says the proposition holds at every preceding stage. The downward-closure monotonicity of $\triangleright\varphi$ requires the `fs_lt_le_lt` axiom to chain a strict predecessor of q through a $\leq$-arrow $q \leq p$ into a strict predecessor of p.

The new definition agrees with the Section 2 version when we instantiate at $(\omega, \leq)$. At depth 0, both definitions give `True`. At depth $n+1$, the new definition says "φ holds at every $m < n+1$", which by downward closure of φ is equivalent to "φ holds at n", exactly the old definition.

Löb's rule, via well-founded induction. The internal Löb rule, in the abstract framework, is a direct appeal to `well_founded_ind` on `fs_lt`:

```
Lemma iLob (φ : iProp) : (▷ φ → φ) ⊢ φ.
Proof.
  intros p. revert p.
  apply (well_founded_ind (fs_lt_wf FS)
                          (fun p => p ⊩ (▷
                              φ → φ) → p
                            ⊩ φ)).
  intros p IH Hp.
  apply Hp; [apply (fs_le_refl FS)|].
  intros p' Hp'.
  apply IH; [exact Hp'|].
  eapply imono; [apply (fs_lt_le FS);
      exact Hp' | exact Hp].
Qed.
```

The proof is almost mechanical given the definitions. We do well-founded induction on p; at each p, the induction hypothesis says that for every strict predecessor p' of p, the implication $\triangleright\varphi \to \varphi$ at p' already gives us φ at p'. We then use `Hp` at p itself — via the `iImpl`'s Kripke condition with `fs_le_refl` to witness $p \leq p$ — which leaves us to prove $\triangleright\varphi$ at p. By the definition of $\triangleright$, this is the quantified statement that φ holds at every strict predecessor p'; for each such p', the induction hypothesis applies (we have `Hp` at p' by downward closure).

This is the precise statement of the equivalence we have been gesturing at: *Löb's rule is well-founded induction on the forcing order, internalized into the logic.* The metatheoretic content is the same; the proof at the meta level uses `well_founded_ind`, and the proof at the iProp level uses `iLob`.

Instances. We exhibit two concrete forcing structures. The first is the natural numbers with their usual order:

```
Definition nat_FS : ForcingStructure :=
  {| fs_cond := nat;
     fs_le := Nat.le;
     fs_lt := Nat.lt;
     fs_le_refl := Nat.le_refl;
     fs_le_trans := Nat.le_trans;
     fs_lt_le := Nat.lt_le_incl;
     fs_lt_le_lt := Nat.lt_le_trans;
     fs_lt_wf := lt_wf
  |}.
```

At `nat_FS`, the abstract framework recovers Section 2. The check `Check iLob nat_FS` confirms the instantiation: the abstract `iLob` applied to `nat_FS` has the type we expect, namely an entailment about Phase 1's `iLater` and `iImpl` (specialized to the natural-number instance) holding in our Phase 1's `iProp`.

The second instance is a non-trivially-different forcing notion: the pointwise product $\omega \times \omega$.

```
Definition le_pair (p q : nat * nat) :
    Prop :=
  fst p ≤ fst q /\ snd p ≤ snd q.

Definition lt_pair (p q : nat * nat) :
    Prop :=
  le_pair p q /\ ~ (fst p = fst q /\ snd p
      = snd q).
```

The order $\leq_{\text{pw}}$ is the componentwise $\leq$; the strict order $<_{\text{pw}}$ is the obvious one (componentwise $\leq$ with inequality in at least one component). This poset is not totally ordered: the pairs $(1, 0)$ and $(0, 1)$ are incomparable. We verify well-foundedness of $<_{\text{pw}}$ by exhibiting the measure $\mathtt{fst}\, p + \mathtt{snd}\, p$ into $\mathbb{N}$, and using the standard library lemma `well_founded_lt_compat`:

```
Lemma lt_pair_wf : well_founded lt_pair.
Proof.
  apply (well_founded_lt_compat _
           (fun p => fst p + snd p)).
  intros p q [[Hf Hs] Hneq].
```

```
    destruct p, q; simpl in *.
    destruct (Nat.eq_dec n n1).
    - subst. destruct (Nat.eq_dec n0 n2).
      + subst. exfalso. apply Hneq. split;
          reflexivity.
      + lia.
    - lia.
  Qed.

  Definition prod_FS : ForcingStructure :=
    {| fs_cond := nat * nat; ... |}.
```

At `prod_FS`, the framework provides a step-indexed logic with *two independent* step counters. A natural application is to model two parallel components that step independently — the step index measures observation depth in each component separately, and Löb's rule lets one reason about recursion that descends in either or both components. We do not develop a Hoare logic over `prod_FS` in this paper, but the existence of the instance is conceptually important: it shows that the framework is not tied to ordinal step-indices in particular, and is in particular applicable to the kind of branching well-founded structure one expects in concurrent settings.

What is recovered. At the abstract level, we have shown that:

1. the data of a step-indexed logic is captured by a `ForcingStructure` record;
2. the propositional fragment of step-indexed Hoare logic, with the Heyting connectives and the later modality, is parametric in this data;
3. the internal Löb rule, in any such structure, is a direct instance of well-founded induction on the strict refinement relation; and
4. the standard $(\omega, \leq)$ choice, used throughout Sections 2–4 and throughout the modern step-indexed literature, is one instance, with at least one non-trivially-different alternative ($\omega \times \omega$ with the pointwise order).

The IMP and λ-calculus developments of Sections 3–4 could be lifted to be parametric in the forcing structure as well, with mostly bookkeeping changes; we have not done so here, for the sake of readability.

7 Well-foundedness is Essential

The abstract framework of Section 6 carries one distinctive axiom: `fs_lt_wf`, the well-foundedness of the strict refinement relation. Every other axiom of `ForcingStructure` is an order-theoretic structural one — reflexivity, transitivity, compatibility. Well-foundedness is the exception, and the only axiom that uses any form of induction principle to state. It is natural to ask whether it is essential: could we drop it and still get a sensible Löb's rule for some other reason?

The answer is no, and we now make this precise. We will exhibit a "would-be" iProp framework over the integers $\mathbb{Z}$, in which the carrier and the order satisfy every part of `ForcingStructure` *except* well-foundedness. All the constructions of Sections 2 and 6 go through: we can build `iPropZ`, `iImplZ`, `iLaterZ`. But Löb's rule fails.

The setup. We work with the integers under $\leq$ and $<$ in the obvious way. Define `iPropZ` as a record of downward-closed predicates on $\mathbb{Z}$:

```
Record iPropZ : Type := MkPropZ {
  iholdZ : Z → Prop;
  imonoZ : forall p q, (q ≤ p)%Z →
      iholdZ p → iholdZ q
}.

Definition ientailsZ (φ ψ : iPropZ) : Prop
    :=
  forall p, iholdZ φ p → iholdZ ψ p.
```

The connectives follow the same definitions as before. We give the ones we need explicitly:

```
Definition iFalseZ : iPropZ :=
  MkPropZ (fun _ => False) (fun _ _ _ H =>
      H).

Definition iImplZ (φ ψ : iPropZ) : iPropZ
    :=
  MkPropZ
    (fun p => forall q, (q ≤ p)%Z →
                        iholdZ φ q →
                            iholdZ ψ q)
    (iImplZ_mono φ ψ).

Definition iLaterZ (φ : iPropZ) : iPropZ
    :=
  MkPropZ
    (fun p => forall p', (p' < p)%Z →
        iholdZ φ p')
    (iLaterZ_mono φ).
```

The definitions are the same as in Section 6, with `fs_cond` := $\mathbb{Z}$, `fs_le` :=$\leq_{\mathbb{Z}}$, and `fs_lt` :=$<_{\mathbb{Z}}$. The order axioms hold: $\leq$ is a preorder, $<$ is contained in $\leq$, and $<$ is compatible with $\leq$ on the right. Only well-foundedness fails: there is no minimum and so no base case for an inductive argument.

The counterexample. Consider $\triangleright\bot$, the iProp asserting that `False` holds at every strict predecessor. We claim this is forced at *no* condition. Indeed, $p \Vdash \triangleright\bot$ would say "for every $p' < p$, $\bot$ holds at p'", i.e., "for every integer strictly less than p, `False` is true". There is at least one such integer, namely $p - 1$, and `False` fails there. So $\triangleright\bot$ is the empty iProp:

```
Lemma lob_premise_at p :
```

```
    p ⊩ (iImplZ (iLaterZ iFalseZ) iFalseZ).
Proof.
  intros q Hqp Hlater.
  apply (Hlater (q - 1)); lia.
Qed.
```

The lemma actually shows something more: at every p, the implication $\triangleright\bot \to \bot$ is forced. The reason is the same as just above. The antecedent $\triangleright\bot$ is empty, so any implication from it is vacuously forced. Spelled out: at p, $p \Vdash (\triangleright\bot \to \bot)$ unfolds to "for every $q \le p$, if $q \Vdash \triangleright\bot$ then $q \Vdash \bot$"; for every q, the antecedent $q \Vdash \triangleright\bot$ is itself false (apply `Hlater` at $q-1$, contradiction), so the conditional is vacuously satisfied.

The conclusion of Löb's rule says $\bot$ is forced at every condition. This obviously fails:

```
Lemma lob_conclusion_fails :
  ~ (forall p, iholdZ iFalseZ p).
Proof. intros H. exact (H 0). Qed.
```

Combining the two: the entailment $(\triangleright\bot \to \bot) \vDash \bot$ is the statement "for every condition, if $(\triangleright\bot \to \bot)$ is forced there then $\bot$ is forced there"; the premise is forced everywhere, the conclusion is forced nowhere, so the entailment fails.

```
Theorem lob_fails_over_Z :
  ~ (iImplZ (iLaterZ iFalseZ) iFalseZ ⊨⊩
      iFalseZ).
Proof.
  intros Hent. apply lob_conclusion_fails.
  intros p. apply Hent. apply
      lob_premise_at.
Qed.
```

This concludes the counterexample: there is a forcing-like structure over $\mathbb{Z}$ in which every other piece of the abstract framework of Section 6 goes through, but Löb's rule is not derivable.

Connection to Gödel–Löb. The result is the step-indexed analog of a classical observation in modal logic. The Gödel–Löb provability logic GL is sound and complete for the class of transitive Kripke frames whose accessibility relation is converse well-founded [Boolos 1993; Smoryński 1985]. Frames where the accessibility relation is *not* converse well-founded — the integers under $<$ are the canonical example — falsify the Löb axiom. Our counterexample is the obvious instantiation of that classical fact into the step-indexed Hoare logic setting: forcing conditions are the worlds, strict refinement is the converse of the accessibility relation, $\triangleright$ is the necessity modality, and Löb's rule is the Löb axiom. Well-foundedness of the strict refinement is exactly converse well-foundedness of the accessibility relation.

Methodological upshot. The "ramification" in ramified forcing is not a presentation convenience to be eliminated. Shoenfield's unramification of set-theoretic forcing showed that for forcing the explicit ordinal hierarchy can be absorbed into a single recursion on rank — because the relevant induction principle is already available at the meta level. In the step-indexed setting, the analogous move is *not* available, because Löb's rule depends on a property of the forcing notion (well-foundedness) that cannot be supplied by any amount of recursion at the meta level: it has to be a property of the conditions themselves. The well-foundedness is constitutive of the modal logic and not eliminable.

The classical literature contains the corresponding result for GL: the soundness of Löb's axiom for converse-well-founded Kripke frames is folklore in modal logic, and Solovay's arithmetical completeness theorem [Solovay 1976] ties it to the well-foundedness of the proof-theoretic "provability in PA" modality. Our contribution is to translate the result into step-indexed Hoare logic — where it amounts to a soundness desideratum for any forcing-like step-indexed semantics — and to mechanize the counterexample. As far as we are aware, the statement in this form has not previously appeared in the PL verification literature.

8 Bridge to Forcing Translations

A parallel literature, alongside the topos-of-trees account of step-indexing surveyed in Section 9.3, develops *syntactic* forcing translations of type theories. Jaber, Tabareau, and Sozeau [Jaber et al. 2012] give a forcing translation of the Calculus of Constructions: for any forcing notion P, they produce a syntactic translation that takes a CIC term $M : T$ to a forced term $M^P : T^P$, where the type translation $T \mapsto T^P$ corresponds to "the P-forced version of T". At the level of propositions, the forced version of Prop is essentially a downward-closed presheaf on the forcing conditions [Jaber et al. 2016] — which is the type of our `iProp`.

The framework of Section 6 is therefore, up to definitional choices, the propositional fragment of the JTS forcing translation of CIC, instantiated at an arbitrary forcing structure. This identification is conceptually clean but not formally mechanized, because we have not formalized the type-level JTS translation in Rocq. What we have mechanized is the *embedding* of the unforced theory (Prop in the ambient metatheory) into the forced one (`iProp` in our framework): the constant embedding. This is the easy half of the correspondence; it makes precise what the forcing translation does to a meta-level proposition that is to be lifted into the forced logic unchanged.

The constant embedding. The embedding $\mathsf{Prop} \hookrightarrow$ `iProp` sends a meta-level proposition P to the iProp that is forced everywhere if P is true and forced nowhere if P is false. Either way it is trivially downward-closed in the forcing order, since it does not depend on the condition:

```
Definition embed (P : Prop) : iProp :=
  MkProp (fun _ => P) (embed_mono P).
```

```
Notation "⌜ P ⌝" := (embed P).
```

This is the trivial part of the forcing translation: classical propositional reasoning lifts to the forcing framework without change. We will see in a moment that the genuinely new content of the forcing translation — the ▷ modality — is *not* in the image of $\ulcorner\cdot\urcorner$.

Commutation with the Heyting connectives. The embedding is a Heyting algebra homomorphism: it commutes with each of $\top$, $\bot$, $\wedge$, $\vee$, $\to$ in both directions of entailment. The proofs are short and structural:

```
Lemma embed_True_l : ⌜ True ⌝ ⊢ iTrue.
Lemma embed_True_r : iTrue ⊢ ⌜ True ⌝.
Lemma embed_False_l : ⌜ False ⌝ ⊢ iFalse.
Lemma embed_False_r : iFalse ⊢ ⌜ False ⌝.
Lemma embed_and_intro P Q : ⌜ P /\ Q ⌝ ⊢ ⌜
    P ⌝ ∧ ⌜ Q ⌝.
Lemma embed_and_elim P Q : ⌜ P ⌝ ∧ ⌜ Q ⌝ ⊢
    ⌜ P /\ Q ⌝.
Lemma embed_or_intro P Q : ⌜ P \/ Q ⌝ ⊢ ⌜
   P ⌝ ∨ ⌜ Q ⌝.
Lemma embed_or_elim P Q : ⌜ P ⌝ ∨ ⌜ Q ⌝ ⊢
   ⌜ P \/ Q ⌝.
Lemma embed_impl_intro P Q : ⌜ P → Q ⌝ ⊢
   (⌜ P ⌝ → ⌜ Q ⌝).
Lemma embed_impl_elim P Q : (⌜ P ⌝ → ⌜ Q ⌝
   ) ⊢ ⌜ P → Q ⌝.
```

The most interesting case is the implication direction $(\ulcorner P\urcorner \to \ulcorner Q\urcorner) \vdash \ulcorner P \to Q\urcorner$, where the Kripke implication of $\to$ has to be discharged by an appeal to the reflexive case "$n \le n$". Given an inhabitant of the Kripke implication at depth n, we apply it at n itself, with the witness of $n \le n$ given by `Nat.le_refl`, and obtain the meta-level implication. The other cases are direct unfoldings of the connectives' Kripke definitions.

Why this matters. Each commutation lemma corresponds to a clause of the propositional fragment of the JTS forcing translation. Specifically, the translation $[\![P]\!]^P$ at the level of propositions commutes with the standard connectives in exactly the way our $\ulcorner\cdot\urcorner$ does — a translated conjunction is the conjunction of translations, and so on [Jaber et al. 2012]. Our framework therefore captures the propositional fragment of the translation. What we do not formalize is the dependent fragment, in which the translation acts non-trivially on universally quantified types and on the universes themselves. That is the part of the correspondence which would, if mechanized, turn the present informal-correspondence claim into a definitional-equality theorem.

The later modality is new content. The constant embedding is not surjective. In particular, its image is not closed under ▷: there is a meta-level proposition P such that $\triangleright\ulcorner P\urcorner$ is not entailment-equivalent to $\ulcorner P\urcorner$. We exhibit the witness:

```
Lemma embed_later_distinct :
  ~ (▷ ⌜ False ⌝ ⊢ ⌜ False ⌝).
Proof.
  intros H. apply (H 0). simpl. trivial.
Qed.
```

At depth 0, $\triangleright\ulcorner\text{False}\urcorner$ is vacuously `True`, whereas $\ulcorner\text{False}\urcorner$ at depth 0 is the constant `False`. The entailment from one to the other therefore fails at depth 0, which is exactly the witness that ▷ produces a proposition outside the embedded image.

This makes precise what the step-indexed-specific content of our framework is. The Heyting algebra structure of `iProp` — all the way through $\top$, $\bot$, $\wedge$, $\vee$, $\to$ — is captured by the embedded image of Prop. What the step-indexed framework adds is the ▷ modality, and the rules that come with it (monotonicity, $\varphi \vdash \triangleright\varphi$, distribution over $\wedge$, and Löb's rule). In the JTS picture, this is the structure that comes from the forcing notion specifically, rather than from the structural machinery of the type theory; in our picture, it is the structure that comes from the well-founded strict refinement of the forcing conditions, rather than from the constant embedding.

A summary correspondence. The picture that emerges is this. Step-indexed Hoare logic, as practiced in Iris and elsewhere, has two layers of structure: a Heyting-algebra fragment, which lifts from the meta-theory by a constant embedding and which therefore has no forcing-specific content; and a modal fragment (▷, `iLob`, the rules around them), which is specifically the structure that the well-founded forcing notion contributes. The JTS forcing translation is the type-theoretic analog of this picture: for any forcing notion P, it gives a translation of CIC that adds P-specific structure to the embedded image of the unforced theory. At the level of propositions, the correspondence we formalize is the constant embedding; a full correspondence at all levels of the universe would require mechanizing the JTS translation itself, which we leave as future work.

9 Related Work

This section is organized into seven literature threads: (i) step-indexing in programming language semantics; (ii) forcing in set theory, ramified and unramified; (iii) topos-theoretic models of step-indexing; (iv) forcing translations in type theory; (v) the modal logic of provability (GL) and well-foundedness; (vi) constructive modal logic for guarded recursion; (vii) predicativity and ramified hierarchies in foundations of mathematics. A final subsection states the gap this paper fills.

9.1 Step-indexing in programming language semantics

Step-indexed logical relations were introduced by Appel and McAllester [Appel and McAllester 2001] to give a semantic account of recursive types in the setting of foundational proof-carrying code. The technique gives a meaning to recursive types not via a fixed point construction in some semantic domain, but by approximating the intended meaning at every finite "observation depth" n. Two terms are equivalent at index n if they cannot be distinguished by any context running for at most n steps; the desired equivalence is then the limit (intersection) of all n-equivalences. Appel and McAllester show that this gives a sound model for recursive types and that the resulting logical relation is preserved under reduction — the canonical *step-down* property that we will recognise as the monotonicity of step-indexed propositions.

Ahmed's PhD thesis [Ahmed 2004] extended step-indexing to languages with mutable references, including general references that can store functions of any type ("higher-order store"). This is the classical setting in which step-indexing is essential rather than merely convenient: the standard set-theoretic recursion on type structure is blocked by the circularity that comes from storing higher-order functions in mutable cells. Ahmed shows how the step-indexed approach absorbs this circularity into the index hierarchy.

Birkedal and collaborators have extended these ideas in two directions. In one direction, *topos-theoretic* reformulations (see Section 9.3 below) replaced the explicit step indices with categorical structure. In the other, the technique was scaled up to richer programming languages and verification settings; see Birkedal et al. [Birkedal et al. 2011] on step-indexed Kripke models for capability calculi.

The current state of the art on the programming-languages side is the *Iris* family of separation logics [Jung et al. 2018; Krebbers et al. 2017]. Iris extends step-indexed separation logic with higher-order ghost state, invariants, and a powerful set of derived rules. Its soundness theorem is mechanized in Coq against a model built ultimately on step-indexed propositions. The internal logic of Iris is itself a step-indexed modal logic with a "later" modality $\triangleright$ and a Löb rule; substantial libraries of Coq tactics support reasoning within it. Other recent step-indexed systems include VST [Appel et al. 2014] and Transfinite Iris [Spies et al. 2021].

Two technical observations from this literature are essential for our work. First, the "later" modality has been a unifying device since at least Appel et al. [Appel et al. 2007], who introduced it as the very-modal $\triangleright$ for guarded recursion. Second, the soundness proof of every step-indexed logic relies on a Löb-style argument — either as an explicit lemma in the internal logic, or implicitly through induction on the step index in the soundness proof. Our paper takes seriously the question of which formulation is more natural in which setting, and argues that for higher-order recursion the internal Löb form is strictly preferable.

9.2 Forcing in set theory: ramified and unramified

Cohen introduced forcing in his 1963 papers [Cohen 1963, 1964] to prove the independence of the continuum hypothesis from ZFC, and gave a book-length treatment in 1966 [Cohen 1966]. The original presentation is what we will call *ramified forcing*: the construction of the generic extension $M[G]$ is stratified by ordinal levels mimicking Gödel's constructible hierarchy. A "ramified language" $\mathcal{L}^*$ has a type symbol for each ordinal α below some sufficient bound, and a variable of type α ranges only over sets at levels strictly below α. The forcing relation $p \Vdash_\alpha \varphi$ is defined level by level: forcing of a level-α formula recurses only on forcing at levels strictly below α.

The two features that make this presentation "ramified" are predicative stratification (a level-α formula only quantifies over levels strictly below α) and explicit level descent (the recursive definition of forcing is structured by ordinal predecessor). Both features are inherited from the ramified theory of types of Russell and Whitehead [Russell and Whitehead 1913] and from the broader tradition of predicative analysis [Feferman 1998; Weyl 1918].

Shoenfield's 1971 lecture [Shoenfield 1971] reformulated forcing without the ramification: forcing conditions and P-names are defined by transfinite recursion on rank, and the forcing relation is defined by recursion on formula complexity, without explicit stratification by syntactic level. Shoenfield's presentation is mathematically equivalent to Cohen's — the same generic extensions are produced — but it is simpler, and it has become the standard modern presentation in textbooks [Jech 2003; Kunen 2011]. Boolean-valued models, due to Scott and Solovay [Scott and Solovay 1967], give yet another equivalent presentation.

What is preserved across these reformulations is the underlying *forcing relation* $p \Vdash \varphi$ together with its fundamental properties. What was eliminated — the explicit ramified syntactic hierarchy — is exactly what we will argue is *not* eliminable in the step-indexed setting.

9.3 Topos-theoretic models of step-indexing

The connection between step-indexing and category-theoretic constructions was made explicit by Birkedal, Møgelberg, Schwinghammer, and Støvring [Birkedal et al. 2012] via the *topos of trees*, the presheaf topos on the partially ordered set $(\omega, \leq)$. They observed that step-indexed propositions are the propositions of the internal logic of this topos, and that the "later" modality $\triangleright$ arises naturally as a particular endofunctor on objects of the topos. Their paper introduces *synthetic guarded domain theory*, in which all of the step-indexed semantics is internalized: rather than indexing manually, one

works in a type theory whose intrinsic semantics is the topos of trees.

The presheaf-topos perspective makes precise much of what the early step-indexed literature handled informally. In particular, the monotonicity condition on step-indexed propositions corresponds exactly to the presheaf condition (functoriality), and the soundness of the Löb rule is the existence of a fixed point of the $\triangleright$ functor (the unique fixed point of $\triangleright X = F(X)$ for a contractive F, or equivalently the well-foundedness of $(\omega, <)$ as the index domain). Subsequent work has refined the categorical setting and used it to model more sophisticated languages [Bizjak et al. 2016].

The presheaf-topos perspective is implicitly a forcing-style semantics in the model-theoretic sense: a presheaf topos on $(P, \leq)$ is exactly the topos of sheaves with respect to the trivial Grothendieck topology on P, and these are exactly the Kripke models of intuitionistic logic with worlds in P. The topos's truth value object plays the role of a Heyting algebra-valued "forcing" truth value. To our knowledge, this connection has not been emphasized in the topos-of-trees literature, and the word "forcing" has not been prominent in it. Our paper makes this terminological and conceptual connection explicit.

9.4 Forcing translations in type theory

A parallel literature, originating in the constructive type theory community, develops syntactic *forcing translations* of type theories. The seminal work here is Jaber, Tabareau, and Sozeau [Jaber et al. 2012], who give a forcing translation of the Calculus of Constructions: given a forcing notion (a poset P), they exhibit a syntactic translation that takes a CIC term $M : T$ to a forced term $M^P : T^P$, where the type translation $T \mapsto T^P$ corresponds to "the forced version of T." For propositions, T^P is a downward-closed predicate on P — exactly our `iProp` of Section 2.

This line was extended in [Jaber et al. 2016] (the "definitional side of the forcing"), which gives a refined translation that preserves more of the computational structure of the original type theory. Coquand and collaborators have related work on constructive presentations of forcing in type-theoretic settings [Coquand and Jaber 2010].

The Jaber et al. translations are the type-theoretic version of ramified forcing for type theory: a parametric construction of a "forced universe" that supports the same operations as the unforced one. Our Phase 4 abstract framework recovers the propositional fragment of this translation by direct construction in Rocq. The Phase 6 file makes the embedding precise via a constant embedding $\mathsf{Prop} \hookrightarrow \mathtt{iProp}$ that is a Heyting algebra homomorphism.

What our work does not do — and what the Jaber et al. line does — is give a translation of the *full* type theory, including dependent types. A natural future direction is to formalize the restriction of the Jaber et al. translation to the propositional fragment in Rocq and check (mechanically) that it produces our `iProp` up to definitional equality.

9.5 Modal logic of provability and well-foundedness

The Gödel–Löb provability logic GL is the modal logic characterized by the axiom $\Box(\Box\varphi \to \varphi) \to \Box\varphi$ (the "Löb axiom"), together with the standard K axioms. Solovay's arithmetical completeness theorem [Solovay 1976] establishes that GL is sound and complete for arithmetical interpretations of $\Box$ as "it is provable in Peano Arithmetic that…". From the model-theoretic side, GL is sound and complete for the class of transitive Kripke frames whose accessibility relation R is *converse well-founded* (no infinite ascending chain $w_0 R w_1 R w_2 R \cdots$); see Smoryński [Smoryński 1985] and Boolos [Boolos 1993] for textbook treatments.

The model theory of GL is directly relevant to our work. The "later" modality $\triangleright$ of step-indexed logic is formally exactly a GL-style provability $\Box$ once we identify the strict refinement order $\prec$ on conditions with the converse of the GL accessibility relation. Löb's rule of step-indexed logic is the Löb axiom of GL. The fact that $\triangleright$ requires well-foundedness for Löb's rule to hold is the dual of GL's requirement that the accessibility relation be converse well-founded. Our Phase 5 result — that Löb fails over $\mathbb{Z}$ — is therefore a special case of (the converse of) the soundness of GL for converse-well-founded frames.

What our paper adds to this picture is the observation that the familiar PL-style step-indexed logic, including its mechanizations in Iris and elsewhere, is in fact a propositional fragment of GL indexed over a particular kind of GL-frame (one with well-founded strict refinement). This connection is folklore in modal logic but, as far as we are aware, has not been made explicit or mechanized for the step-indexed Hoare logic setting.

9.6 Constructive modal logic and guarded recursion

A separate line of work, with substantial overlap with synthetic guarded domain theory, develops constructive modal logics for guarded recursion. Atkey and McBride [Atkey and McBride 2013] use the $\triangleright$ modality to characterize *productive coprogramming*: a corecursive function is productive iff each recursive call sits under a $\triangleright$. Clouston, Bizjak, Grathwohl, and Birkedal [Clouston et al. 2015, 2017] develop guarded λ-calculi with $\triangleright$ as a primitive type former, supporting recursive types via a guarded fixed-point operator. Møgelberg and others [Bizjak and Møgelberg 2018; Møgelberg 2014] extend these systems to dependent types and provide categorical semantics.

This literature is closely allied with synthetic guarded domain theory: both treat $\triangleright$ as a built-in modal operator and exploit its compatibility with recursion. The work is structured mostly around *computational* and *type-theoretic* questions

("how do we type recursion productively?"), rather than *semantic* questions ("what does the step-indexed semantics look like as a forcing construction?"). Our paper is complementary: we work on the semantic side, with the forcing reading as our organizing principle.

9.7 Predicativity and ramified hierarchies

The word "ramified" belongs to a tradition older than Cohen's forcing. Russell and Whitehead [Russell and Whitehead 1913] introduced the ramified theory of types in *Principia Mathematica* to resolve the set-theoretic paradoxes via a stratification of propositions by "order," where a proposition of order n may quantify only over propositions of order $< n$. The same idea animates Weyl's predicative analysis [Weyl 1918], in which the real numbers are constructed within an arithmetical hierarchy without appeal to impredicative set-theoretic principles. Feferman's program of *predicativity given the natural numbers* [Feferman 1998] develops this into a comprehensive foundation for substantial parts of classical mathematics.

The thread connecting ramified type theory to ramified forcing is that both stratify quantification by a well-founded hierarchy, and both treat the hierarchy itself as part of the syntactic apparatus rather than as a meta-level convenience. Shoenfield's unramification of forcing showed that for forcing the stratification can be absorbed into a single recursion on rank; what we argue, by mechanizing step-indexed Hoare logic in higher-order settings, is that for step-indexing the analogous unramification *cannot* be performed. The well-founded forcing notion is doing real work, not just structuring the proof.

This connection also clarifies why the Iris-style internal logic "feels predicative": step-indexed propositions are stratified by a well-founded hierarchy, and any recursive definition that does not respect the hierarchy — by mediating recursion through a $\triangleright$ — is rejected. The system is not just predicative in spirit; it is precisely a ramified predicative system in the technical sense.

9.8 Hoare logic and program verification

We omit a survey of classical Hoare logic and separation logic, referring the reader to standard sources [Brookes 2007; Hoare 1969; O'Hearn et al. 2001]. Two points are worth noting. First, our IMP development in Section 3 uses the plain (non-separation) version of Hoare logic, since separation machinery is not needed for the conceptual point we want to make about the `while` rule. Second, Cook's relative completeness theorem [Cook 1978] for Hoare logic shows that the classical proof system is as complete as can be hoped given any reasonable specification language; our results are orthogonal to this — we study how the soundness side is best organized, not the proof system.

9.9 The gap this paper fills

The summaries above suggest that almost every ingredient of our work is, individually, well-known to specialists in some adjacent community. The topos-of-trees perspective on step-indexing is familiar in synthetic guarded domain theory; the GL modal logic of provability is familiar to modal logicians; the ramified-forcing presentation of Cohen is familiar to set theorists; the JTS forcing translation is familiar to the type theory community; and the predicative reading of ramification is familiar to foundations researchers.

What does *not* appear in the literature, to our knowledge, is a single mechanized account of step-indexed Hoare logic that (i) frames step-indexing explicitly as ramified forcing, making the connection to Cohen's original presentation precise rather than implicit in the topos-theoretic literature; (ii) demonstrates that internal Löb is the load-bearing soundness tool for higher-order recursion by mechanizing both an IMP-style and a λ-calculus-with-`fix` Hoare logic and exhibiting both the meta-level and internal forms of the `while` rule's soundness proof; (iii) generalizes the framework to abstract forcing notions parametrically, exhibiting at least one second instance and recovering the standard $(\omega, \leq)$ instance; (iv) proves the well-foundedness of the forcing notion is essential by an explicit counterexample over $\mathbb{Z}$ where the iProp constructions go through but Löb's rule fails; (v) verifies the bridge to the JTS forcing translation by formalizing the constant embedding $\texttt{Prop} \hookrightarrow \texttt{iProp}$ as a Heyting algebra homomorphism and identifying $\triangleright$ as the step-indexed-specific addition.

The contribution is therefore not the introduction of any single new construction but the *integration* of several existing threads into one mechanized account, plus the small new technical result of Section 7 and the explicit comparison of Sections 3 and 4 that diagnoses where internal Löb earns its keep.

10 Conclusion

We have given a forcing-style semantics for step-indexed Hoare logic, mechanized in Rocq across the propositional layer, an IMP Hoare logic, and an untyped λ-calculus with general recursion. The forcing reading places the step-indexed technique in direct correspondence with Cohen's ramified forcing construction: the natural numbers are the forcing conditions, the later modality $\triangleright$ is the level-descent operator, and Löb's rule is well-founded induction on the forcing order, internalized into the logic.

What we found. Two observations stand out. First, in the first-order setting of IMP, the `while` rule admits two structurally identical soundness proofs: an outer induction on the step index, and an appeal to the internal Löb rule. The two proofs have the same content because at the meta level, Löb's rule *is* induction on the step index. Mechanizing both

side by side, as we do, makes the equivalence concrete rather than analogical.

Second, in the higher-order setting of the untyped λ-calculus with `fix`, the analogous soundness proof has a different shape. The soundness predicate for a recursive function is itself defined in terms of the function being verified, and unrolling this self-reference at the meta level produces a non-trivial induction that does not arise from the program's step count alone. The natural proof, which we mechanize, invokes the internal Löb rule directly: "show that under the assumption $\triangleright\varphi$ we have φ, then conclude φ by Löb." No meta-level induction on a step count is needed. The structural difference between the IMP and λ-fix proofs is a difference in the *recursion locus* of the soundness predicate (Section 5): for IMP, the predicate is recursive only in the step index, and outer induction on n matches its structure; for λ-fix, the predicate is recursive in the program itself, and internal Löb is the natural tool that plays to that structure.

Three further observations. The abstract framework of Section 6 makes precise that the choice of ω as the step-index domain is a *computational* convenience rather than a foundational one. Any well-founded forcing notion supports the same internal logic, with $\triangleright$ defined uniformly as "forces at every strict predecessor" and Löb derived by well-founded induction. We exhibit the pointwise product $\omega\times\omega$ as a second instance, but the framework is generic.

Section 7 shows that the well-foundedness cannot be dropped: we construct a non-well-founded forcing notion over $\mathbb{Z}$ in which Löb's rule fails. This is a special case of the model theory of GL, but the statement for step-indexed Hoare logic does not, as far as we are aware, appear in the verification literature. The claim has methodological weight: step-indexed Hoare logic is not just "Hoare logic with a step counter"; it requires a well-founded forcing notion to be sound at all.

Section 8 verifies that the iProp framework is the propositional fragment of the Jaber–Tabareau–Sozeau forcing translation of CIC, via a constant embedding $\mathsf{Prop} \hookrightarrow \mathtt{iProp}$ that we prove to be a Heyting algebra homomorphism. The $\triangleright$ modality is the genuinely new content of iProp over the embedded image, and arises uniformly from the well-founded strict refinement on conditions.

Limitations. The Hoare logic for the λ-calculus in Section 4 is, at present, restricted to partial correctness against a single specification predicate S that serves as both pre- and post-condition. The fix soundness rule `wp_fix` (Section 4) accordingly handles only this case. A more general rule with distinct pre/post-conditions, a rule for fix at higher types, and an integration with separation logic on a heap are all natural directions but lie outside the present development.

The bridge to JTS forcing translations of Section 8 formalizes only the constant embedding. A full mechanization of the type-level translation, against an explicit forcing structure and with the propositional rules verified to match our `iProp`, is the natural next step.

The abstract framework of Section 6 parametrizes the propositional layer over arbitrary well-founded forcing structures, but the IMP and λ-calculus developments of Sections 3–4 are presented at the specific $(\omega, \leq)$ instance for clarity. Lifting them to be parametric in the forcing structure is straightforward and not done here.

Future directions. Several extensions suggest themselves.

- *Mechanizing the full JTS translation* (or a useful fragment) and verifying that its propositional fragment coincides definitionally with our `iProp` at the corresponding forcing structure. This would convert the heuristic correspondence of Section 8 into a theorem.
- *Step-indexing over richer forcing structures* — transfinite ordinals (cf. Transfinite Iris [Spies et al. 2021]), branching conditions for genuinely concurrent step counters, or mixed structures combining several step-index dimensions. The abstract framework already supports these in principle; the work is to exhibit non-trivial verifications at the new instances.
- *Lifting the Hoare-logic developments to the abstract framework*, so that IMP and the λ-calculus admit verifications parametric in the forcing structure. This is straightforward but bookkeeping-heavy; it would make the parametric story of Section 6 the primary presentation rather than an afterthought.
- *An integration with separation logic* along the lines of Iris, lifting the forcing reading from the propositional logic to the resource-aware extension. This is a substantial undertaking but would let the forcing perspective speak directly to state-of-the-art verification.

The broader point we hope the paper makes is that step-indexed reasoning, viewed as a forcing construction, sits at the intersection of three research traditions that do not usually talk to each other: set-theoretic forcing, modal logic of provability, and step-indexed semantics for verification. Each tradition contains piece-of-the-picture insights that the other two could profit from. The mechanization that accompanies this paper is intended both as evidence for the technical claims and as a starting point for others wishing to explore the intersection further.

References

Amal Ahmed. 2004. *Semantics of Types for Mutable State.* Ph. D. Dissertation. Princeton University.

Andrew W. Appel et al. 2014. *Program Logics for Certified Compilers.* Cambridge University Press. The VST (Verified Software Toolchain) framework..

Andrew W. Appel and David A. McAllester. 2001. An Indexed Model of Recursive Types for Foundational Proof-Carrying Code. *ACM Trans. Program. Lang. Syst.* 23, 5 (2001), 657–683.

Andrew W. Appel, Paul-André Melliès, Christopher D. Richards, and Jerome Vouillon. 2007. A Very Modal Model of a Modern, Major, General Type System. *ACM SIGPLAN Notices (POPL)* 42, 1 (2007), 109–122. Often referred to as "the very-modal model"; introduces ▷ for step-indexed logical relations..

Robert Atkey and Conor McBride. 2013. Productive Coprogramming With Guarded Recursion. In *ICFP*. 197–208.

Lars Birkedal, Rasmus Ejlers Møgelberg, Jan Schwinghammer, and Kristian Støvring. 2012. First Steps in Synthetic Guarded Domain Theory: Step-Indexing in the Topos of Trees. *Logical Methods in Computer Science* 8, 4 (2012).

Lars Birkedal, Bernhard Reus, Jan Schwinghammer, Kristian Støvring, Jacob Thamsborg, and Hongseok Yang. 2011. Step-Indexed Kripke Models over Recursive Worlds. In *POPL*. 119–132.

Aleš Bizjak, Hans Bugge Grathwohl, Ranald Clouston, Rasmus Ejlers Møgelberg, and Lars Birkedal. 2016. Guarded Dependent Type Theory With Coinductive Types. In *FoSSaCS*. 20–35.

Aleš Bizjak and Rasmus Ejlers Møgelberg. 2018. Denotational Semantics for Guarded Dependent Type Theory. In *FoSSaCS*.

George S. Boolos. 1993. *The Logic of Provability*. Cambridge University Press.

Stephen Brookes. 2007. A Semantics for Concurrent Separation Logic. *Theoretical Computer Science* 375, 1–3 (2007), 227–270.

Ranald Clouston, Aleš Bizjak, Hans Bugge Grathwohl, and Lars Birkedal. 2015. Programming and Reasoning With Guarded Recursion for Coinductive Types. In *FoSSaCS*. 407–421.

Ranald Clouston, Aleš Bizjak, Hans Bugge Grathwohl, and Lars Birkedal. 2017. The Guarded Lambda-Calculus: Programming and Reasoning With Guarded Recursion for Coinductive Types. *Logical Methods in Computer Science* 12, 3 (2017).

Paul J. Cohen. 1963. The Independence of the Continuum Hypothesis. *Proc. National Academy of Sciences* 50 (1963), 1143–1148.

Paul J. Cohen. 1964. The Independence of the Continuum Hypothesis, II. *Proc. National Academy of Sciences* 51 (1964), 105–110.

Paul J. Cohen. 1966. *Set Theory and the Continuum Hypothesis*. W. A. Benjamin.

Stephen A. Cook. 1978. Soundness and Completeness of an Axiom System for Program Verification. *SIAM J. Comput.* 7, 1 (1978), 70–90.

Thierry Coquand and Guilhem Jaber. 2010. A Note on Forcing and Type Theory. *Fundamenta Informaticae* 100 (2010), 43–52.

Solomon Feferman. 1998. *In the Light of Logic*. Oxford University Press.

C. A. R. Hoare. 1969. An Axiomatic Basis for Computer Programming. *Commun. ACM* 12, 10 (1969), 576–580.

Guilhem Jaber, Gabriel Lewertowski, Pierre-Marie Pédrot, Matthieu Sozeau, and Nicolas Tabareau. 2016. The Definitional Side of the Forcing. In *LICS*. 367–376.

Guilhem Jaber, Nicolas Tabareau, and Matthieu Sozeau. 2012. Extending Type Theory With Forcing. In *LICS*. 395–404.

Thomas Jech. 2003. *Set Theory* (3rd ed.). Springer.

Ralf Jung, Robbert Krebbers, Jacques-Henri Jourdan, Aleš Bizjak, Lars Birkedal, and Derek Dreyer. 2018. Iris From the Ground Up: A Modular Foundation for Higher-Order Concurrent Separation Logic. *Journal of Functional Programming* 28 (2018).

Robbert Krebbers, Ralf Jung, Aleš Bizjak, Jacques-Henri Jourdan, Derek Dreyer, and Lars Birkedal. 2017. The Essence of Higher-Order Concurrent Separation Logic. In *ESOP*. 696–723.

Kenneth Kunen. 2011. *Set Theory*. College Publications.

Rasmus Ejlers Møgelberg. 2014. A Type Theory for Productive Coprogramming Via Guarded Recursion. In *CSL-LICS*.

Peter W. O'Hearn, John C. Reynolds, and Hongseok Yang. 2001. Local Reasoning About Programs That Alter Data Structures. *CSL* (2001).

Bertrand Russell and Alfred N. Whitehead. 1910–1913. *Principia Mathematica*. Cambridge University Press.

Dana Scott and Robert M. Solovay. 1967. Boolean-Valued Models for Set Theory. In *Proc. American Math. Soc. Summer Inst. on Axiomatic Set Theory*. Unpublished; Boolean-valued model presentation of forcing..

Joseph R. Shoenfield. 1971. Unramified Forcing. *Axiomatic Set Theory, Proc. Symp. Pure Math.* 13 (1971), 357–381.

Craig Smoryński. 1985. *Self-Reference and Modal Logic*. Springer.

Robert M. Solovay. 1976. Provability Interpretations of Modal Logic. *Israel J. Math.* 25 (1976), 287–304.

Simon Spies, Niklas Mück, Hoang-Hai Dang, Manuel Rigger, Michael Sammler, Joseph Tassarotti, Robbert Krebbers, and Derek Dreyer. 2021. Transfinite Iris: Resolving an Existential Dilemma of Step-Indexed Separation Logic. *PLDI* (2021). Extends Iris with transfinite step indices..

Hermann Weyl. 1918. *Das Kontinuum*. Veit, Leipzig. Translated as "The Continuum: A Critical Examination of the Foundation of Analysis," Dover, 1994..